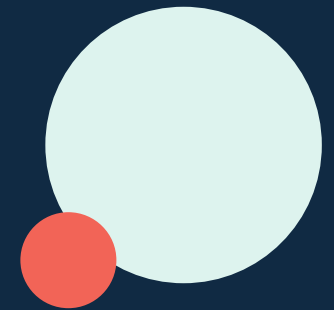


Sistema de Gestão de Produtos

Como o código foi organizado, quais decisões sustentam a solução e onde ela pode evoluir.

PYTHON 3.10+

CLI + CSV



O problema que o sistema resolve

Uma aplicação de terminal para manter um catálogo pequeno, local e persistente.

- Autenticar o operador antes de liberar o acesso.
- Cadastrar produtos com identificação, preço, quantidade e categoria.
- Consultar, alterar parcialmente e remover registros.
- Manter os dados entre execuções usando um arquivo CSV.

Escopo escolhido

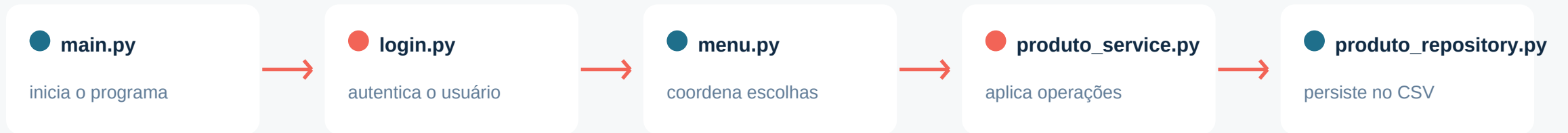
Uma solução simples e didática: poucos módulos, biblioteca padrão e armazenamento sem banco de dados.

Entrada

```
python main.py
```

Cada módulo tem uma responsabilidade

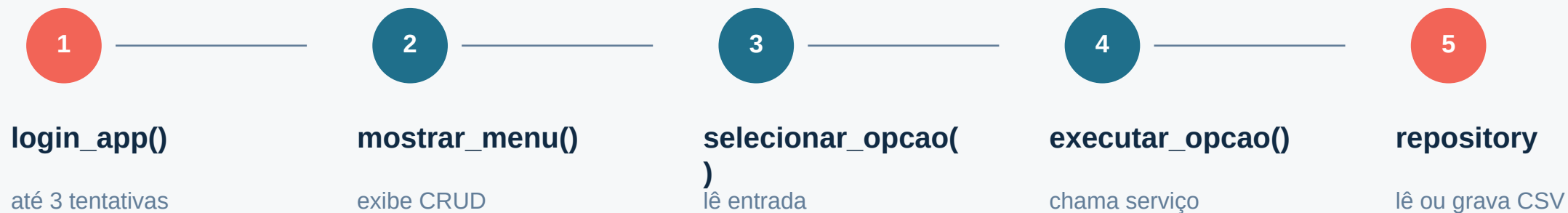
A separação reduz acoplamento e torna o fluxo mais fácil de entender e testar.



A entidade Produto fica no centro: representa o dado e protege regras como categoria válida e atualização de timestamps.

Do login à operação escolhida

O programa permanece em um laço até o usuário escolher sair.



```
while True:
    mostrar_menu()
    opcao = selecionar_opcao()
    if not executar_opcao(opcao):
        break
```

Produto é a unidade de negócio

A classe usa dataclass para concentrar dados e comportamento relacionado ao produto.

```
@dataclass(slots=True)
class Produto:
    id: int
    nome: str
    quantidade: int
    preco: Decimal
    categoria: str | Categoria
    data_criacao: datetime
    data_ultima_alteracao: datetime | None
```

Decimal

Evita tratar preço como float e deixa a representação monetária mais

Enum

Categoria.validar() normaliza e rejeita valores fora do conjunto permitido.

Atualização parcial

Enter mantém o valor atual; atualizar_dados() altera apenas os campos recebidos e marca a última alteração.

CRUD com regras próximas do uso

O módulo de serviço traduz entradas do terminal em operações sobre ProdutoRepository.

CREATE / cadastrar_produto()

Converte entradas, valida ID duplicado e salva.

READ / listar_produtos()

Busca a lista e monta uma tabela textual.

UPDATE / atualizar_produto()

Localiza por ID e permite alterar campos individualmente.

DELETE / remover_produto()

Localiza, remove e persiste novamente.

A camada recebe o repository por parâmetro: isso facilita trocar a persistência em testes ou em uma evolução futura.

O CSV funciona como uma pequena base local

O repositório esconde detalhes de arquivo para o restante do sistema.

```
CSV -> DictReader -> Produto  
Produto -> DictWriter -> CSV
```

- Ao iniciar, carrega registros e converte tipos: int, Decimal e datetime.
- Ao salvar ou remover, reescreve o arquivo completo com cabeçalho e ISO dates.
- Cria a pasta pai automaticamente quando necessário.

Por que CSV?

Baixa complexidade, fácil inspeção manual e suficiente para um catálogo pequeno/local.

Uma sessão típica no terminal

O comportamento esperado pode ser entendido acompanhando uma única jornada do usuário.

```
$ python main.py

=== LOGIN ===

Usuário: admin

Senha: 1234

Acesso liberado.

1 - Cadastrar    2 - Consultar

Escolha uma opção: 1

Produto cadastrado com sucesso.
```

O que acontece por trás

menu.py cria o repository; o serviço coleta e valida; Produto representa; repository persiste.

Por que esse exemplo importa

Ele conecta as funções abstratas do código ao resultado que o operador vê na tela.

Decisões que tornam o código compreensível

A estrutura privilegia legibilidade e separação de responsabilidades para o tamanho do problema.

Biblioteca padrão

Menos dependências e instalação simples.

Repository

Isola leitura e escrita do CSV.

Dataclass

Declara o modelo com pouco código repetido.

Enum de categorias

Transforma valores livres em conjunto conhecido.

Injeção do repository

Permite substituir a persistência com baixo impacto.

Laço principal

Mantém a interação ativa até a opção 0.

O que eu melhoraria em uma próxima versão

O projeto cumpre o escopo didático; estes são os próximos ganhos de robustez.

- **Segurança** Retirar credenciais do código e usar hash/variáveis de ambiente.
- **Validação** Impedir ID, quantidade e preço negativos; exigir nome não vazio.
- **Coesão** Transformar operações do service em métodos da classe ou remover a classe não usada.
- **Confiabilidade** Tratar CSV inválido e usar escrita temporária/atômica.
- **Testes** Cobrir login, categoria, CRUD e conversões do repository.
- **Caminho do arquivo** Resolver o CSV relativo à raiz do projeto, não ao diretório atual.

Mensagem final

A arquitetura já separa as decisões principais; a próxima etapa é fortalecer as bordas.

Obrigado

Sistema de Gestão de Produtos

Código-fonte: main.py • Execução: python main.py